LAB 3 — PROBLEM STATEMENT #43

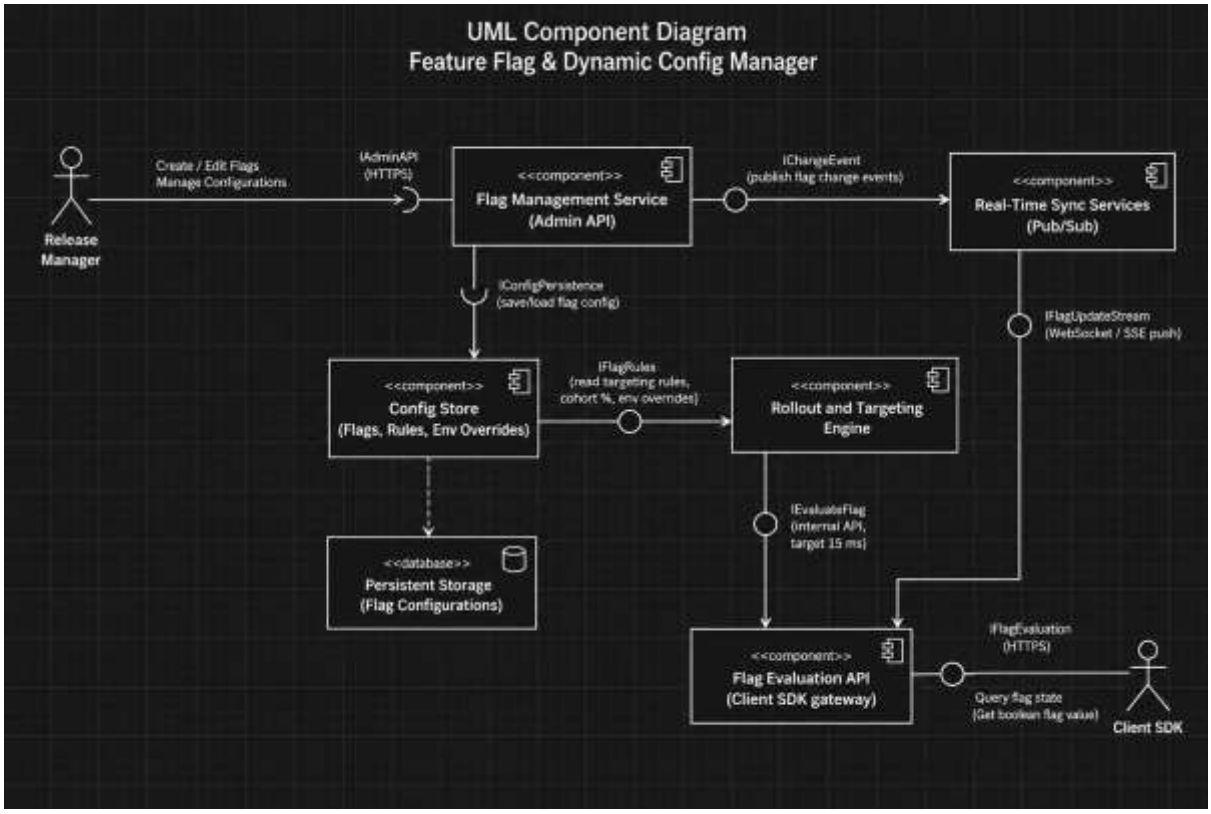
Feature Flag & Dynamic Config Manager

The Problem #43 scenario describes a centralized configuration and feature-toggling platform supporting:

- Percentage-based user cohort rollouts
- Environment overrides
- Real-time client SDK flag synchronization

The target actors are **Software Engineer** and **Release Manager**.

UML Component Diagram



Written Architectural Justification

Architectural Selection: Microservices Architecture

We chose Microservices Architecture for the Feature Flag & Dynamic Config Manager.

Architectural Choice

Microservices Architecture is selected because the Feature Flag & Dynamic Config Manager consists of multiple logically independent functions, including feature flag management, configuration storage, targeting and rollout evaluation, flag evaluation, and real-time synchronization with client SDKs. These functions can be separated into individual services with clearly defined interfaces.

Reason 1 — Independent Scaling

Feature flag evaluation can receive a large number of requests from client applications. With Microservices Architecture, the **Flag Evaluation API** can be scaled independently according to its workload without requiring all other services to be scaled. This is suitable for a centralized feature flag platform supporting multiple client applications.

Reason 2 — Independent Deployment and Fault Isolation

The services can be developed and deployed independently. For example, changes to the **Rollout & Targeting Engine** can be made without redeploying the entire platform. Separating services also provides better **fault isolation**, meaning a failure in one service can be contained rather than directly affecting every part of the system.

Security Advantage

Microservices Architecture allows access to individual services to be controlled according to their responsibilities. Administrative operations such as creating or modifying feature flags can be restricted to authorized users such as Software Engineers and Release Managers, while client applications can be given limited access to flag evaluation and synchronization services.

Performance Benefit

The **Flag Evaluation API** can be independently optimized and scaled to handle frequent flag evaluation requests. This helps maintain low evaluation latency and supports the requirement that the feature flag evaluation API execute in **under 15 ms**, minimizing overhead when integrated into client critical paths.

Conclusion

Therefore, Microservices Architecture is appropriate for the Feature Flag & Dynamic Config Manager because it supports independent scaling, independent deployment, fault isolation, controlled service access, and efficient feature flag evaluation and synchronization.